PROYECTO INTEGRADOR

Arquitectura Orientada a Servicios

8 entregas semanales · Desarrollo progresivo de una solución real

Carrera	Ingeniería en Software
Materia	Arquitectura Orientada a Servicios
Docente	Ing. Luis Leonardo Mendoza Reséndiz
Equipo	_____
Periodo	_____

1. Propósito del proyecto

Durante ocho semanas, cada equipo desarrollará de forma incremental una solución de software basada en principios de Arquitectura Orientada a Servicios. El proyecto deberá partir de un problema real o verosímil, proponer una solución tecnológica clara y evolucionar desde el análisis inicial hasta una implementación funcional, documentada y demostrable.

2. Objetivo general

Diseñar, construir, integrar y documentar una solución de software orientada a servicios que resuelva un problema identificado por el equipo, aplicando análisis de requisitos, diseño de arquitectura, APIs REST, persistencia de datos, seguridad, pruebas y documentación técnica.

3. Modalidad de trabajo

- Trabajo en equipos de 3 a 5 integrantes.
- Una entrega por semana durante 8 semanas.
- Todas las entregas son acumulativas.
- Cada equipo deberá utilizar un repositorio de control de versiones.
- Cada integrante deberá poder explicar y defender cualquier parte del proyecto.
- El profesor podrá solicitar correcciones antes de considerar una entrega como aceptada.

4. Reglas generales

1. El proyecto debe resolver un problema concreto; no se aceptarán ideas demasiado genéricas sin usuario, contexto o necesidad definida.
2. Las decisiones técnicas deberán justificarse. No basta con indicar una tecnología: se debe explicar por qué se utiliza.
3. Toda funcionalidad reportada como terminada debe poder demostrarse.
4. Las evidencias deben corresponder al proyecto del equipo.

5. El código debe mantenerse organizado y documentado.
6. La entrega final debe integrar lo trabajado durante las ocho semanas; no es un proyecto distinto.

5. Ruta de entregas

Semana	Entrega	Resultado principal	Tipo	Ponderación sugerida
1	Propuesta del proyecto	Problema, solución, alcance y objetivos	Documento	10%
2	Análisis y arquitectura	Requisitos, servicios y diagrama de arquitectura	Documento + diagrama	10%
3	Diseño de API	Recursos, endpoints y contratos	Documento técnico	10%
4	Modelo de datos	Modelo ER, esquema y scripts SQL	Documento + SQL	10%
5	MVP Backend I	Estructura del backend y primeros CRUD	Código + evidencia	15%

Semana	Entrega	Resultado principal	Tipo	Ponderación sugerida
6	MVP Backend II	Integración, reglas de negocio, seguridad y errores	Código + Postman	15%
7	Calidad y documentación	Pruebas, OpenAPI, seguridad y despliegue	Código + documentación	15%
8	Entrega final	Sistema integrado, demo y defensa técnica	Proyecto + presentación	15%

ENTREGA 1 — Propuesta del problema y solución

Semana 1 · Documento de propuesta

La primera entrega define el proyecto completo. El equipo deberá identificar un problema real o realista, describir a quién afecta y proponer una solución de software viable. En esta etapa todavía no se evalúa la implementación; se evalúa la calidad del problema, la claridad de la propuesta y la factibilidad del proyecto.

Contenido mínimo del documento

- Portada: nombre del proyecto, materia, equipo e integrantes.
- Nombre provisional del sistema.
- Contexto del problema: dónde ocurre, quiénes participan y qué proceso existe actualmente.
- Descripción del problema: qué sucede, por qué es importante y qué consecuencias provoca.
- Usuarios o actores afectados.
- Objetivo general de la solución.
- Mínimo 3 objetivos específicos.
- Descripción de la solución propuesta.
- Alcance inicial: qué incluirá el proyecto.
- Fuera de alcance: qué no realizará esta versión.
- Funciones principales esperadas.
- Beneficios esperados.
- Riesgos iniciales y limitaciones.
- Tecnologías preliminares consideradas.
- Fuentes o referencias utilizadas para comprender el problema.

Preguntas que debe responder el equipo

- ¿Qué problema estamos resolviendo?
- ¿Quién necesita esta solución?
- ¿Cómo se resuelve actualmente?
- ¿Por qué vale la pena desarrollar el sistema?
- ¿Qué cambiaría si nuestra solución funciona correctamente?
- ¿Qué sería una primera versión viable del proyecto?

Entregables obligatorios

- ☐ Documento PDF/DOCX de propuesta.
- ☐ Diagrama simple del proceso actual (AS-IS).
- ☐ Diagrama simple de la solución propuesta (TO-BE).
- ☐ Lista preliminar de funciones.
- ☐ Presentación oral de 3 a 5 minutos.

Criterios mínimos de aceptación

- El problema está claramente definido y no se limita a describir una aplicación.
- Existe una relación directa entre problema y solución.
- El alcance puede desarrollarse durante las ocho semanas.
- Los integrantes pueden explicar la idea sin leer el documento.

ENTREGA 2 — Análisis de requisitos y arquitectura

Semana 2 · Del problema a una arquitectura orientada a servicios

En esta entrega el equipo convierte la propuesta en un diseño técnico inicial. Debe identificar qué necesita hacer el sistema, qué cualidades deberá cumplir y cómo dividirá la solución en servicios.

Trabajo requerido

- Revisar y corregir la propuesta de la semana 1.
- Identificar requisitos funcionales.
- Identificar requisitos no funcionales.
- Definir actores y permisos iniciales.
- Identificar entidades principales del dominio.
- Identificar servicios o módulos principales.
- Definir responsabilidad de cada servicio.
- Indicar qué servicios se comunican entre sí.
- Proponer el stack tecnológico.
- Crear un diagrama de arquitectura inicial.

Requisitos mínimos

Tipo	Cantidad mínima	Ejemplos
Funcionales	10	Registrar usuario, crear pedido, consultar historial
No funcionales	5	Seguridad, disponibilidad, rendimiento, usabilidad
Actores/Roles	2	Cliente, administrador, profesor, operador
Servicios/Módulos	4	Usuarios, autenticación, pagos, notificaciones

Entregables obligatorios

- ☐ Documento actualizado con requisitos.
- ☐ Tabla de requisitos funcionales y no funcionales.
- ☐ Lista de actores/roles.
- ☐ Catálogo de servicios con responsabilidad de cada uno.
- ☐ Diagrama de arquitectura SOA.
- ☐ Justificación del stack tecnológico.

Criterios mínimos de aceptación

- Los servicios tienen responsabilidades claras y no duplicadas.
- La arquitectura corresponde al problema planteado.
- Los requisitos pueden verificarse.
- El equipo puede explicar por qué dividió el sistema de esa manera.

ENTREGA 3 — Diseño de la API y contratos de servicio Semana 3 · Definición de cómo se comunicarán los componentes

El objetivo es definir de forma precisa cómo serán consumidos los servicios. En esta semana todavía puede utilizarse documentación y ejemplos sin que todo el backend esté implementado.

Trabajo requerido

• Identificar los recursos principales de la API.
• Diseñar rutas REST claras y consistentes.
• Asignar métodos HTTP correctos.
• Definir Path Parameters y Query Parameters.
• Definir Request Body y Response Body.
• Definir códigos HTTP de éxito y error.
• Definir un formato estándar de errores.
• Definir autenticación prevista.
• Definir versionado inicial de la API.

Mínimos técnicos

Elemento	Mínimo
Recursos	5
Endpoints documentados	20
Ejemplos JSON de request	5
Ejemplos JSON de response	5
Códigos HTTP explicados	8
Casos de error	5

Entregables obligatorios

☐ Catálogo de endpoints.
☐ Contratos JSON.
☐ Tabla de códigos HTTP.
☐ Ejemplos de errores.
☐ Diagrama de flujo de al menos una operación completa.
☐ Primer borrador OpenAPI/Swagger opcional pero recomendado.

Criterios mínimos de aceptación

• Los endpoints representan recursos y no acciones arbitrarias.
• GET, POST, PUT/PATCH y DELETE se utilizan de forma coherente.

• Los contratos permiten comprender qué debe enviar y recibir un consumidor.
• La API contempla errores, no solo casos exitosos.

ENTREGA 4 — Modelo de datos y persistencia

Semana 4 · Diseño de la información que utilizará el sistema

El equipo deberá convertir las necesidades del sistema en un modelo de datos consistente y preparar la persistencia que utilizará el backend.

Trabajo requerido

• Identificar entidades y atributos.
• Definir llaves primarias.
• Definir llaves foráneas y relaciones.
• Aplicar reglas básicas de normalización.
• Crear el diagrama entidad-relación.
• Crear el esquema físico de la base de datos.
• Preparar scripts de creación.
• Preparar datos de prueba.
• Explicar qué servicio utiliza cada grupo de datos.
• Definir restricciones importantes: UNIQUE, NOT NULL, CHECK o equivalentes.

Mínimos técnicos

• Mínimo 5 tablas o colecciones, salvo justificación.
• Relaciones claramente representadas.
• Script reproducible para crear la base de datos.
• Mínimo 10 registros de prueba distribuidos entre las entidades principales.
• Consultas básicas para verificar el funcionamiento.

Entregables obligatorios

☐ Diagrama ER.
☐ Diccionario de datos.
☐ Script SQL de creación.
☐ Script o archivo de datos de prueba.
☐ Documento que relacione servicios con datos.
☐ Capturas o evidencia de la base de datos funcionando.

Criterios mínimos de aceptación

• El modelo soporta las operaciones definidas en la API.
• No existen relaciones imposibles o ambiguas.
• Los scripts pueden ejecutarse desde cero.
• El equipo comprende el significado de cada tabla y relación.

ENTREGA 5 — MVP Backend I

Semana 5 · Primera implementación funcional

Esta semana comienza la implementación. El objetivo es demostrar que la arquitectura definida puede convertirse en software funcional.

Implementación mínima

• Proyecto backend configurado.
• Estructura organizada por rutas, controladores, servicios y acceso a datos (o equivalente). • Variables de entorno para configuración sensible.
• Conexión funcional con la base de datos.
• CRUD completo de al menos 2 recursos principales.
• Validaciones básicas.
• Manejo de 404 y 500.
• Colección inicial de Postman.
• Repositorio actualizado con README de ejecución.

Evidencias requeridas

• Crear un registro.
• Consultar uno y varios registros.
• Modificar un registro.
• Eliminar un registro.
• Mostrar al menos un error de validación.
• Mostrar al menos un recurso inexistente.

Entregables obligatorios

- ☐ Código fuente en repositorio.
- ☐ Archivo README con instalación y ejecución.
- ☐ Colección Postman.
- ☐ Evidencias de los CRUD.
- ☐ Actualización del documento técnico si hubo cambios de arquitectura.

Criterios mínimos de aceptación

• El proyecto puede ejecutarse siguiendo el README.
• Los endpoints demostrados funcionan contra datos persistentes.
• No todo el código está concentrado en un único archivo sin justificación.
• Los cambios respecto al diseño anterior están documentados.

ENTREGA 6 — MVP Backend II e integración

Semana 6 · Reglas de negocio, seguridad y comunicación

El backend deberá evolucionar de CRUD básicos a una solución con lógica de negocio, control de acceso e integración entre componentes.

Implementación mínima

• Agregar los recursos principales restantes.
• Implementar autenticación.

• Implementar al menos 2 roles o niveles de autorización.
• Aplicar validaciones de negocio.
• Definir y usar manejo consistente de errores.
• Implementar una operación que involucre más de un recurso o servicio.
• Utilizar correctamente transacciones o estrategias de consistencia cuando sea necesario.
• Agregar filtros, búsqueda o paginación en al menos un endpoint.
• Actualizar la colección Postman.

Escenario obligatorio

El equipo debe demostrar un flujo completo de negocio. Por ejemplo: usuario inicia sesión → crea una operación → el sistema valida permisos → consulta o modifica información relacionada → genera respuesta final. El flujo debe corresponder al problema del proyecto.

Entregables obligatorios

☐ Backend actualizado.
☐ Flujo completo demostrado.
☐ Autenticación y autorización funcionales.
☐ Colección Postman con casos positivos y negativos.
☐ Documento corto de reglas de negocio implementadas.

Criterios mínimos de aceptación

• Un usuario sin permisos no puede ejecutar operaciones protegidas.
• La lógica importante no depende únicamente del cliente.
• Los errores regresan respuestas entendibles.
• Existe al menos un flujo que conecte varias partes del sistema.

ENTREGA 7 — Pruebas, seguridad, documentación y despliegue Semana 7 · Preparación para una entrega profesional

El sistema deberá prepararse para ser evaluado como producto. El objetivo no es agregar muchas funciones nuevas, sino demostrar calidad, seguridad básica, documentación y estabilidad.

Trabajo requerido

• Crear pruebas de endpoints o lógica crítica.
• Probar casos correctos e incorrectos.
• Revisar validación de entradas.
• Revisar almacenamiento de contraseñas y datos sensibles.
• Revisar variables de entorno y secretos.
• Agregar documentación OpenAPI/Swagger o equivalente.
• Completar el README.
• Preparar instrucciones para despliegue o ejecución.
• Realizar una prueba básica de carga o concurrencia sobre al menos un endpoint.
• Registrar defectos encontrados y correcciones realizadas.

Checklist básico de seguridad

☐ No existen contraseñas en texto plano en la base de datos.
☐ No existen secretos reales dentro del repositorio.
☐ Los endpoints protegidos verifican autenticación.
☐ Los endpoints sensibles verifican autorización.
☐ Las entradas del usuario se validan.
☐ Los errores no revelan información interna innecesaria.
☐ Existe manejo de CORS acorde al proyecto.
☐ La API no acepta operaciones imposibles sin validación.

Entregables obligatorios

☐ Plan y evidencia de pruebas.
☐ Documentación de API.
☐ Checklist de seguridad.
☐ README final preliminar.
☐ Evidencia de despliegue o procedimiento reproducible.
☐ Lista de errores encontrados y corregidos.

Criterios mínimos de aceptación

• Otra persona puede comprender la API sin leer el código fuente.
• Las pruebas incluyen errores y permisos, no solo happy path.
• La configuración sensible está separada del código.
• El sistema puede ejecutarse o desplegarse siguiendo instrucciones.

ENTREGA 8 — Entrega final y defensa del proyecto

Semana 8 · Sistema integrado, presentación y demostración

La última entrega integra todo lo realizado. El equipo deberá presentar una versión estable del sistema, demostrar los flujos principales y defender las decisiones técnicas tomadas durante las ocho semanas.

Contenido final obligatorio

• Problema y solución final.
• Requisitos finales.
• Arquitectura final.
• Servicios implementados.
• Modelo de datos final.
• API documentada.
• Sistema funcional.
• Seguridad y permisos.
• Pruebas realizadas.
• Instrucciones de ejecución/despliegue.
• Limitaciones conocidas.
• Mejoras futuras.

Demostración mínima

• Inicio de sesión o identificación del usuario.
• Al menos un flujo principal completo.
• Operación CRUD relevante.
• Regla de negocio.
• Control de permisos.
• Manejo de un error real.
• Persistencia en la base de datos.

Defensa técnica

El profesor podrá pedir a cualquier integrante que explique:

• Por qué eligieron esa arquitectura.
• Cómo se comunican los componentes.
• Por qué existe cada servicio.
• Cómo se protege un endpoint.
• Cómo se almacena la información.
• Qué sucede cuando ocurre un error.
• Qué cambiarían si el sistema tuviera 100 veces más usuarios.
• Qué parte del proyecto mejoraría con más tiempo.

Entregables obligatorios

☐ Repositorio final.
☐ Documento técnico final.
☐ Presentación.
☐ Colección Postman o equivalente.
☐ Script y/o respaldo de base de datos.
☐ Documentación OpenAPI/Swagger.
☐ Video o demo en vivo, según indicaciones del profesor.
☐ Retrospectiva del equipo.

Criterios mínimos de aceptación

• La versión presentada corresponde al repositorio entregado.
• Los flujos principales funcionan.
• El equipo puede demostrar y explicar su trabajo.
• La documentación permite reproducir el proyecto.

6. Evaluación sugerida del proyecto

La siguiente ponderación distribuye mayor valor en las etapas de implementación y cierre. El docente puede ajustar los porcentajes según el calendario del curso.

Entrega	Descripción	Valor
1	Propuesta	10%
2	Análisis y arquitectura	10%

3	Diseño de API	10%
4	Modelo de datos	10%
5	MVP Backend I	15%
6	MVP Backend II	15%
7	Calidad y documentación	15%
8	Entrega final	15%

7. Criterios transversales

Criterio	Qué se espera
Cumplimiento	Se entrega lo solicitado y puede demostrarse.
Justificación técnica	Las decisiones tienen una razón clara.
Calidad	Código, documentación y diagramas son comprensibles.
Evolución	Se atiende la retroalimentación de semanas anteriores.
Trabajo en equipo	Los integrantes conocen el proyecto y contribuyen.
Defensa	El equipo puede explicar su arquitectura y su implementación.

8. Formato de cada entrega semanal

Cada semana, junto con los archivos técnicos, el equipo deberá incluir una portada o reporte breve con:

- ☐ Número y nombre de la entrega.
- ☐ Nombre del proyecto.
- ☐ Integrantes.
- ☐ Objetivo de la semana.
- ☐ Qué se implementó o diseñó.
- ☐ Qué cambió respecto a la entrega anterior.
- ☐ Problemas encontrados.
- ☐ Cómo se resolvieron.
- ☐ Pendientes para la siguiente semana.
- ☐ Enlace al repositorio y versión/tag o commit de la entrega.

9. Bitácora de cambios

Semana	Cambio realizado	Motivo	Responsable(s)	Evidencia

10. Lista de verificación final

☐ El problema original sigue siendo claro y la solución lo atiende.
☐ La arquitectura final está actualizada.
☐ Los servicios implementados corresponden al diseño final.
☐ La base de datos puede crearse desde los scripts entregados.
☐ Los endpoints importantes están documentados.
☐ Existen validaciones y manejo de errores.
☐ Existen autenticación y permisos cuando el sistema lo requiere.
☐ El proyecto puede ejecutarse siguiendo el README.
☐ No se entregan secretos o contraseñas reales.
☐ Las pruebas principales están documentadas.
☐ El repositorio está ordenado.
☐ La presentación coincide con el sistema real.
☐ Todos los integrantes están preparados para la defensa.

Importante: una entrega semanal no se considera aislada. Si un elemento anterior cambia, el documento, la arquitectura, los contratos y el código deberán mantenerse sincronizados.

11. Espacio para aprobación del proyecto

Nombre del proyecto	
Equipo	
Problema aprobado	Sí / No / Con observaciones
Alcance aprobado	Sí / No / Con observaciones
Observaciones del docente	
Firma / Fecha	